

JavaScript

Functions vs Arrow Functions

A beginner-friendly guide to understanding the difference between regular functions and arrow functions
— written in simple English.

1. What is a Regular Function?

A **regular function** (also called a **normal function** or **function declaration**) is the classic way to write a function in JavaScript. You use the **function** keyword to create it.

```
// Regular Function
function greet(name) {
  return "Hello, " + name;
}

greet("Raj"); // Output: Hello, Raj
```

Regular functions can also be written as **function expressions** (stored in a variable):

```
// Function Expression
const greet = function(name) {
  return "Hello, " + name;
};
```

2. What is an Arrow Function?

An **arrow function** is a shorter, modern way to write functions. It was introduced in **ES6 (2015)**. Instead of the **function** keyword, you use a special arrow symbol: **=>**

```
// Arrow Function
const greet = (name) => {
  return "Hello, " + name;
};

greet("Raj"); // Output: Hello, Raj
```

If the function body has **only one line**, you can make it even shorter — no curly braces, no return keyword needed:

```
// One-liner Arrow Function
const greet = name => "Hello, " + name;
```

3. Side-by-Side Comparison

Feature	Regular Function	Arrow Function
Syntax	function name() {}	const name = () => {}
this keyword	Has its OWN 'this'	Borrows 'this' from outside
arguments object	Available ✓	Not available ✕
Hoisting	Hoisted (can call before defining)	Not hoisted
Used as constructor	Yes (new keyword works)	No (cannot use new)
Best used for	Methods in objects, complex logic	Callbacks, short operations

4. The 'this' Keyword — The Biggest Difference

The word **this** in JavaScript refers to the object that is currently running the code. This is where regular and arrow functions behave very differently.

Problem with regular function:

```
const person = {
  name: "Raj",
  greet: function() {
    setTimeout(function() {
      // 'this' here is NOT person – it lost the context!
      console.log("Hello " + this.name); // undefined
    }, 1000);
  }
};
```

Fixed with arrow function:

```
const person = {
  name: "Raj",
  greet: function() {
    setTimeout(() => {
      // Arrow function keeps 'this' from outside (person)
      console.log("Hello " + this.name); // Hello Raj ✓
    }, 1000);
  }
};
```

■ **Tip:** Arrow functions do NOT have their own 'this'. They automatically use the 'this' from the surrounding code. This makes them perfect for callbacks inside methods.

5. Advantages of Arrow Functions

Advantage	What it means
■ Shorter syntax	Less code to write — easier to read
■ No 'this' confusion	Perfect for callbacks — no need to use .bind() or var self = this
■ Great for arrays	Works beautifully with .map(), .filter(), .reduce()
■ Implicit return	One-liners don't need the return keyword
■ Cleaner callbacks	Event listeners and async code look much cleaner

6. Real-World Examples

Example 1 — Double all numbers in an array:

```
const numbers = [1, 2, 3, 4, 5];

// Regular function way
const doubled1 = numbers.map(function(n) {
  return n * 2;
});

// Arrow function way (much shorter!)
const doubled2 = numbers.map(n => n * 2);

console.log(doubled2); // [2, 4, 6, 8, 10]
```

Example 2 — Filter even numbers:

```
const nums = [1, 2, 3, 4, 5, 6];

const evens = nums.filter(n => n % 2 === 0);

console.log(evens); // [2, 4, 6]
```

Example 3 — Add numbers (with two params):

```
const add = (a, b) => a + b;

console.log(add(3, 7)); // 10
```

7. When NOT to Use Arrow Functions

Arrow functions are great, but they don't work everywhere. Avoid them in these situations:

■	Object methods	When you need 'this' to refer to the object itself
---	-----------------------	--

■	Constructors	You cannot use 'new' with arrow functions
■	Event handlers needing 'this'	When 'this' should point to the DOM element
■	Functions using 'arguments'	Arrow functions don't have the 'arguments' object

8. Quick Syntax Cheat Sheet

Situation	Arrow Function Syntax
No parameters	<code>const fn = () => 'hello';</code>
One parameter	<code>const fn = x => x * 2;</code>
Two+ parameters	<code>const fn = (x, y) => x + y;</code>
Multiple lines	<code>const fn = (x) => { let r = x*2; return r; };</code>
Return an object	<code>const fn = x => ({ value: x });</code>

9. Summary — Which One Should You Use?

Use Regular Function when...	Use Arrow Function when...
You need 'this' to refer to the calling object	Writing short, one-line operations
You are creating a constructor (with new)	Using callbacks like <code>.map()</code> , <code>.filter()</code> , <code>.reduce()</code>
You use the 'arguments' object	Inside event listeners (without needing 'this')
You want hoisting (call before defining)	Async/await and Promise chains

Remember: Arrow functions are not a full replacement for regular functions — they are a tool. Use the right one for the right job! Happy coding ■